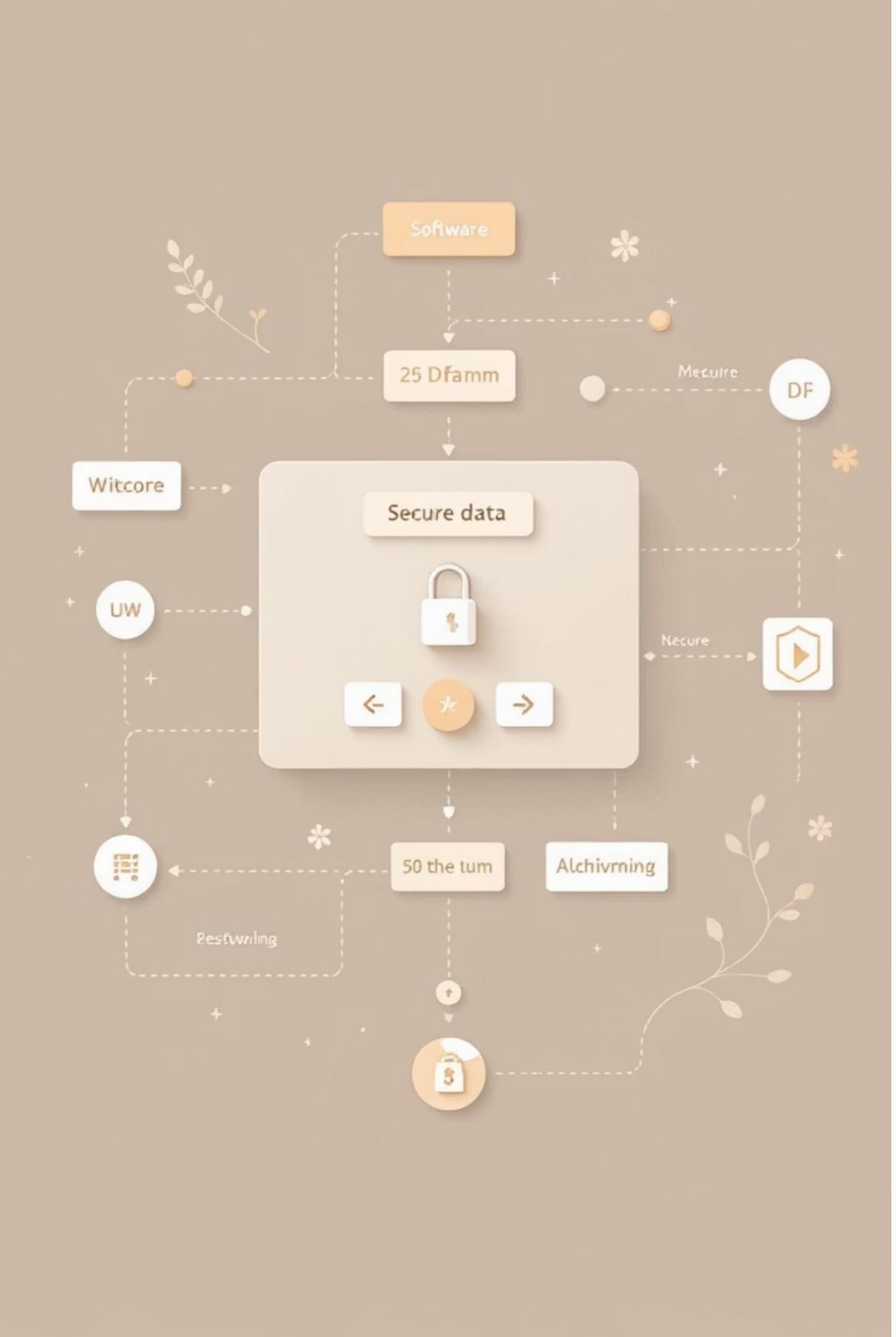
Diseño de Gestor de Contactos “MI AGENDA”

Liliana Vásquez-García

liliana,.vasquez@gmx.es

Esta presentación explora las decisiones de diseño y las soluciones implementadas en el desarrollo de un gestor de contactos.

Arquitectura, los desafíos superados y los logros clave para construir un sistema eficiente y fácil de usar.



ARQUITECTURA DEL PROYECTO

Fundamentos Estructurales y de Datos

Modelo de Datos

Se utilizó una clase `Contacto` que incorpora principios de **Encapsulación**. Los atributos se protegen mediante decoradores `@property` y `setters`, indicando privacidad con el prefijo `_nombre`.

Estructuras Híbridas

El `GestorDeContactos` combina una **Lista** para iteraciones eficientes y un **Diccionario** para búsquedas rápidas por nombre, optimizando el rendimiento general del sistema.

Desafíos Clave en el Desarrollo

El desarrollo de un sistema robusto y fácil de usar siempre presenta obstáculos. Aquí se detalla los principales desafíos enfrentados y cómo se abordaron para asegurar la integridad y funcionalidad del gestor.

Validación de Duplicados

Asegurar la unicidad de los contactos fue crítico, enfocándonos en evitar dos entradas con el mismo nombre o teléfono (duplicados).

Sincronización de Datos

Mantener la `lista_de_contactos` y el `diccionario_de_contactos` coherentes durante las operaciones de edición y eliminación.

Manejo de Errores

Implementar mecanismos para prevenir fallos (crashes) cuando los usuarios introducían datos no válidos o inexistentes. Se muestran mensajes explicicativs para el usuario.

```python
def agregar_contacto(self, nombre, telefono, email, direccion):
    if nombre in self.diccionario_de_contactos:
        raise ValueError(f"El contacto '{nombre}' ya existe")

    for contacto in self.lista_de_contactos:
        if contacto.telefono == telefono:
            raise ValueError(f"El número {telefono} ya se encuentra asociado a otro contacto")
```

```python
    # Busquedas de contactos
    def buscar_por_contacto(self, nombre):
        if nombre not in self.diccionario_de_contactos:
            raise ValueError("Este contacto no existe")
        return self.diccionario_de_contactos[nombre]

    def buscar_por_telefono(self, telefono):
        for contacto in self.lista_de_contactos:
            if contacto.telefono == telefono:
                return contacto
        raise ValueError("Este número no se encuentra registrado")
```

SOLUCIONES IMPLEMENTADAS

Estrategias para la Estabilidad y Eficiencia

01

Excepciones Personalizadas Personalizadas

Se utilizó `try-except` en el menú principal para interceptar `ValueErrors` del gestor, mostrando mensajes amigables en lugar de errores técnicos.

02

List Comprehension

Se aplicó para la eliminación eficiente de objetos en la lista, como en `[c for c in self.lista_de_contactos if c.nombre != nombre]`.

03

Pre-carga de Datos

Un diccionario `contactos_existentes` inicializa el sistema con 10 ejemplos reales, facilitando pruebas y evaluaciones inmediatas.

```python
mi_agenda = GestorDeContactos()

for nombre, datos in contactos_existentes.items():
    try:
        mi_agenda.agregar_contacto(nombre, datos['telefono'], datos['email'], datos['direccion'])
    except ValueError as e:
        print(f"Error al cargar {nombre}: {e}")
```

```python
if nuevo_telefono:
    for c in self.lista_de_contactos:
        if c.telefono == nuevo_telefono and c.nombre != nombre_actual:
            print(f"Error: El teléfono {nuevo_telefono} ya lo tiene otro contacto.")
            return
    contacto_encontrado.telefono = nuevo_telefono
```

```python
contactos_existentes = {
    "Ana García": {"telefono": "09723456789", "email": "anitagort@gmail.com", "direccion": "Julio Zaldumbide 345"},
    "Carlos Rodríguez": {"telefono": "09876543210", "email": "carlosrodri@gmail.com", "direccion": "Av. Alameda 123"},
    "Ariana Rosero": {"telefono": "09876544367", "email": "arianarori@gmail.com", "direccion": "Av. 10 de Julio 578"},
    "Luis Martínez": {"telefono": "09912345678", "email": "luismartz@gmail.com", "direccion": "Calle Larga 789"},
    "Elena Vizcaíno": {"telefono": "09655443322", "email": "elenaviz@gmail.com", "direccion": "Av. de los Shyris N45"},
    "Ricardo Torres": {"telefono": "095887766555", "email": "rtorres99@gmail.com", "direccion": "Pasaje San Luis 102"},
    "Sofía Méndez": {"telefono": "09811223344", "email": "sofimendez@gmail.com", "direccion": "Barrio Las Orquídeas 56"},
    "Javier Espinoza": {"telefono": "09766554433", "email": "jespinoza_dev@gmail.com", "direccion": "Av. Colón 1240"},
    "Valeria Castro": {"telefono": "09933445566", "email": "valecast@gmail.com", "direccion": "Urbanización El Bosque 12"},
    "Diego Paredes": {"telefono": "09899887766", "email": "dparedes_7@gmail.com", "direccion": "Calle de los Olivos 404"}
}
```

LOGROS OBTENIDOS

Un Sistema Robusto y Modular

```
Hola! Bienvenida/o a tu Agenda de Contactos


Selecciona el número de la opción que desees realizar:
1. Agregar contacto
2. Buscar contacto por nombre
3. Buscar contacto por número de teléfono
4. Editar contacto exixtente
5. Eliminar contacto
0. Salir
```

PARTE 1: modelo de contactos - programación orientada a objetos (POO)
Parte 2: gestion de contactos
Parte 3: diccionario con contactos para prueba
Parte 4: creación de instancia y agregar los contactos de prueba
Parte 5: ejecución y pruebas

```
4. Editar contacto exixtente
5. Eliminar contacto
0. Salir


Opción: 4
Nombre del contacto que desea editar: Valeria Castro

Datos actuales de Valeria Castro:
Teléfono: 09933445566
Email: valecast@gmail.com
Dirección: Urbanización El Bosque 12

--- Ingrese los nuevos datos (deje en blanco para no cambiar) ---
Nuevo teléfono: 1515615
Nuevo email:
Nueva dirección: Los consquistadores
```

Interfaz Robusta

Un menú interactivo intuitivo que guía al usuario y previene activamente la entrada de datos erróneos.

Código Modular

La lógica de datos (Modelo) está claramente separada de la lógica de control (Gestor) y de la interfaz de usuario (Menú).

Flexibilidad de Edición

El método de edición permite actualizar selectivamente campos específicos (teléfono, email, dirección) sin alterar el resto de la información.

Encapsulación: Protegiendo los Datos

La encapsulación es un pilar fundamental de la programación orientada a objetos. En nuestro gestor, la clase `Contacto` ejemplifica este principio, asegurando que los datos internos permanezcan consistentes y sean manipulados de manera controlada.

- **Atributos Privados:** Uso de `_nombre`, `_telefono`, etc., para indicar que estas variables son de uso interno.

- **Métodos de Acceso:** `@property` permite acceder a los atributos como si fueran públicos, pero internamente gestiona su lectura.

- **Métodos de Modificación:** Los `setters` (`@nombre.setter`) controlan cómo se asignan nuevos valores, permitiendo validaciones antes de la actualización.

```python
# Definición de getters de datos y aplicación de rpincipios de encapsulación

    @property
    def nombre(self):
        return self._nombre

    @property
    def telefono(self):
        return self._telefono

    @property
    def email(self):
        return self._email

    @property
    def direccion(self):
        return self._direccion

# Definición de edición de datos

    @nombre.setter
    def nombre(self, nombre):
        self._nombre = nombre

    @telefono.setter
    def telefono(self, telefono):
        self._telefono = telefono

    @email.setter
    def email(self, email):
        self._email = email
```

Optimizando el Rendimiento con Estructuras Híbridas

La elección de las estructuras de datos es crucial para la eficiencia. El `GestorDeContactos` emplea una combinación estratégica de lista y diccionario para maximizar tanto la iterabilidad como la velocidad de búsqueda.

Lista para Iteración

Ideal para recorrer todos los contactos cuando se necesita mostrar un listado completo o aplicar una operación a cada elemento.

Diccionario para Búsqueda

Permite búsquedas de contactos por nombre en tiempo casi constante, crucial para operaciones rápidas como la edición o eliminación.

Sincronización Constante

Asegura que ambas estructuras reflejen siempre el estado actual de los datos, garantizando consistencia.

```
Opción: 1
Nombre: Valeria Castro
Teléfono: 1564333
Email: valek@gmail.com
Dirección: Las perdices 56
No se pudo guardar: El contacto 'Valeria Castro' ya existe
```

```
Opción: 2
Nombre del contacto a buscar: Juanita Miranda

Este contacto no existe
```

```
Opción: 3
Número de teléfono del contacto a buscar: 4886525

Este número no se encuentra registrado
```

```
Selecciona el número de la opción que desees realizar:
1. Agregar contacto
2. Buscar contacto por nombre
3. Buscar contacto por número de teléfono
4. Editar contacto exixtente
5. Eliminar contacto
0. Salir


Opción: 8
Opción inválida. Por favor, selecciona un número mencionado en el menú.


Selecciona el número de la opción que desees realizar:
1. Agregar contacto
2. Buscar contacto por nombre
3. Buscar contacto por número de teléfono
4. Editar contacto exixtente
5. Eliminar contacto
0. Salir


Opción:
```

Manejo Inteligente de Errores

Un sistema robusto no solo funciona bien, sino que también maneja los errores con elegancia. Implementamos un sistema de manejo de errores que mejora la experiencia del usuario y la estabilidad del programa.

Entrada Inesperada

El usuario introduce un nombre de contacto que no existe o un dato en formato incorrecto

Detección con `try-except`

El bloque `try-except` captura el `ValueError` lanzado por la lógica de negocio.

Mensaje Amigable

En lugar de un *crash*, se muestra un mensaje claro al usuario, explicando el problema.

Programa Continúa

El flujo de la aplicación no se interrumpe, manteniendo una experiencia fluida.

Diseño Modular para Mantenibilidad

La separación de responsabilidades es clave para un código limpio y mantenible. Nuestro gestor sigue este principio dividiendo claramente la aplicación en capas distintas.

```
Hola! Bienvenida/o a tu Agenda de Contactos


Selecciona el número de la opción que desees realizar:
1. Agregar contacto
2. Buscar contacto por nombre
3. Buscar contacto por número de teléfono
4. Editar contacto exixtente
5. Eliminar contacto
0. Salir
```

Interfaz de Usuario (Menú)

Maneja la interacción con el usuario, la entrada y salida de datos, y presenta las opciones disponibles.

```python
### PARTE 2: GESTION DE CONTACTOS

# Almacenamiento de contactos a través de listas y diccionarios
class GestorDeContactos:
    def __init__(self):
        self.lista_de_contactos = []
        self.diccionario_de_contactos = {}

    def agregar_contacto(self, nombre, telefono, email, direccion):
        if nombre in self.diccionario_de_contactos:
            raise ValueError(f"El contacto '{nombre}' ya existe")

        for contacto in self.lista_de_contactos:
            if contacto.telefono == telefono:
                raise ValueError(f"El número {telefono} ya se encuentra asociado a otro contacto")
```

Lógica de Control (Gestor)

Actúa como intermediario, orquestando las operaciones y gestionando el flujo de trabajo entre la interfaz y el modelo de datos.

```python
class Contacto:
    def __init__(self, nombre, telefono, email, direccion):
        self.nombre = nombre
        self.telefono = telefono
        self.email = email
        self.direccion = direccion
```

Modelo de Datos (Contacto)

Encargado de la representación, almacenamiento y validación de la información de los contactos.

Esta modularidad facilita la depuración, la extensión de funcionalidades y la colaboración entre desarrolladores.

Conclusiones y Próximos Pasos Pasos

El desarrollo de este gestor de contactos ha sido una experiencia valiosa en la que se desarrolló y aplicacó herramientas básicas de codificación, diseño de software y resolución de problemas.

Aprendizajes Clave

La importancia de la encapsulación, la elección adecuada de estructuras de datos y el manejo de errores son fundamentales.

Mejoras Futuras

Búsqueda por múltiples criterios, y una interfaz gráfica de usuario.